Before we look at atomic operations, there's one more concept that's important to understand about objects and memory locations: modification orders.

5.1.3. Modification orders

Every object in a C++ program has a defined *modification order* composed of all the writes to that object from all threads in the program, starting with the object's initialization. In most cases this order will vary between runs, but in any given execution of the program all threads in the system must agree on the order. If the object in question isn't one of the atomic types described in section 5.2, you're responsible for making certain that there's sufficient synchronization to ensure that threads agree on the modification order of each variable. If different threads see distinct sequences of values for a single variable, you have a data race and undefined behavior (see section 5.1.2). If you do use atomic operations, the compiler is responsible for ensuring that the necessary synchronization is in place.

This requirement means that certain kinds of speculative execution aren't permitted, because once a thread has seen a particular entry in the modification order, subsequent reads from that thread must return later values, and subsequent writes from that thread to that object must occur later in the modification order. Also, a read of an object that follows a write to that object in the same thread must either return the value written or another value that occurs later in the modification order of that object. Although all threads must agree on the modification orders of each individual object in a program, they don't necessarily have to agree on the relative order of operations on separate objects. See section 5.3.3 for more on the ordering of operations between threads.

So, what constitutes an atomic operation, and how can these be used to enforce ordering?

5.2. Atomic operations and types in C++

An *atomic operation* is an indivisible operation. You can't observe such an operation half-done from any thread in the system; it's either done or not done. If the load operation that reads the value of an object is *atomic*, and all modifications to that object are also *atomic*, that load will retrieve either the initial value of the object or the value stored by one of the modifications.

The flip side of this is that a nonatomic operation might be seen as half-done by another thread. If that operation is a store, the value observed by another thread might be neither the value before the store nor the value stored but something else. If the nonatomic operation is a load, it might retrieve part of the object, have another thread modify the value, and then retrieve the remainder of the object, thus retrieving neither the first value nor the second but some combination of the two. This is a simple problematic race condition, as described in chapter 3, but at this level it may constitute a *data race* (see section 5.1) and thus cause undefined behavior.

In C++, you need to use an atomic type to get an atomic operation in most cases, so let's look at those.